

Verifying JUnit in the Development Environment

A practical guide to confirming your test setup works correctly

 by Rodrigo Martins Pagliares

Last update: 11/25/2025



Verifying JUnit integration with Gradle in the Development Environment



01

After adding required JUnit libraries, tests can run directly in the IDE

02

Create a new test class under `src/test/java` (e.g., **DemoTest**)

03

Add a simple test method to confirm JUnit is configured correctly

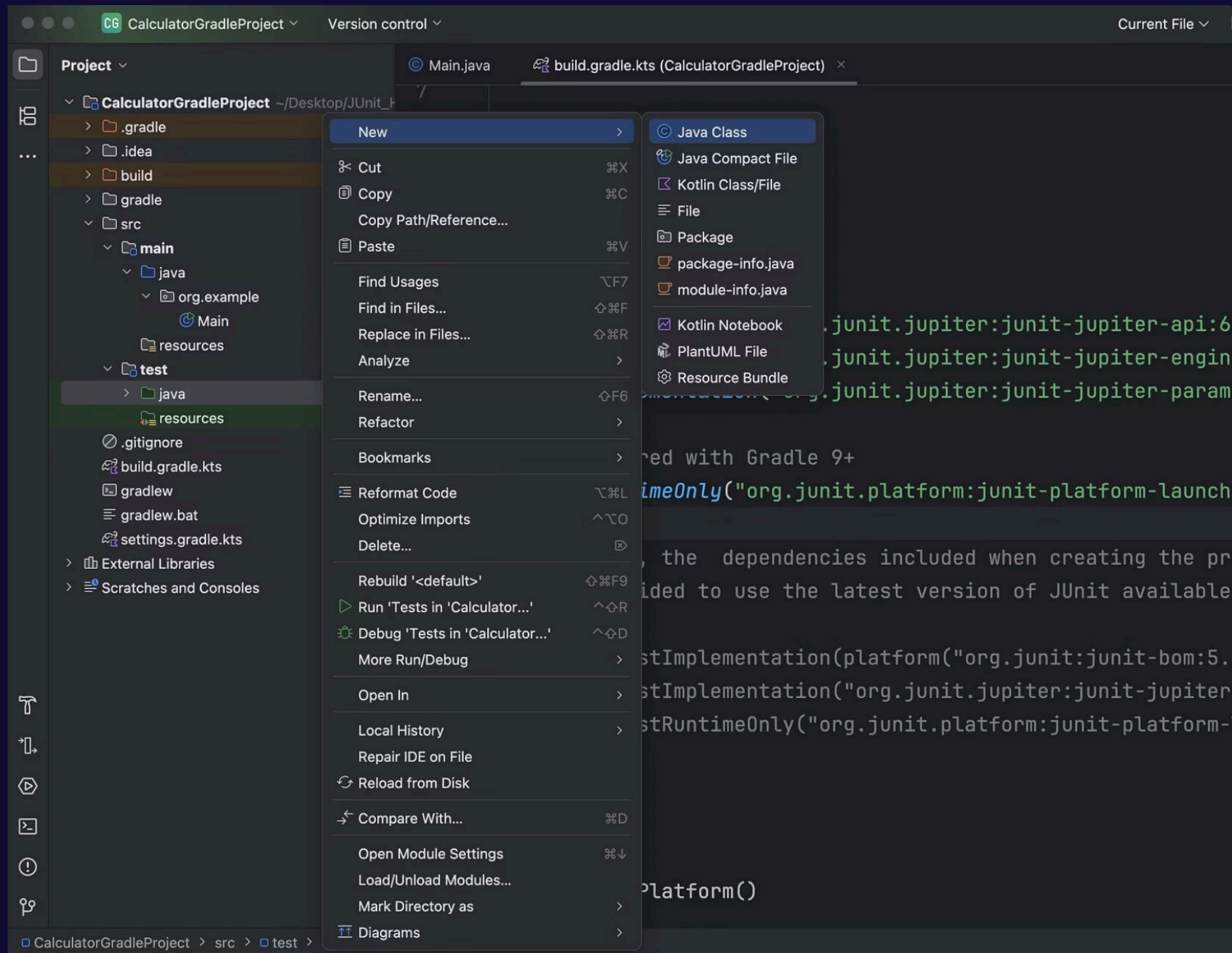
04

Use the green **Run** (play) icon to execute the test

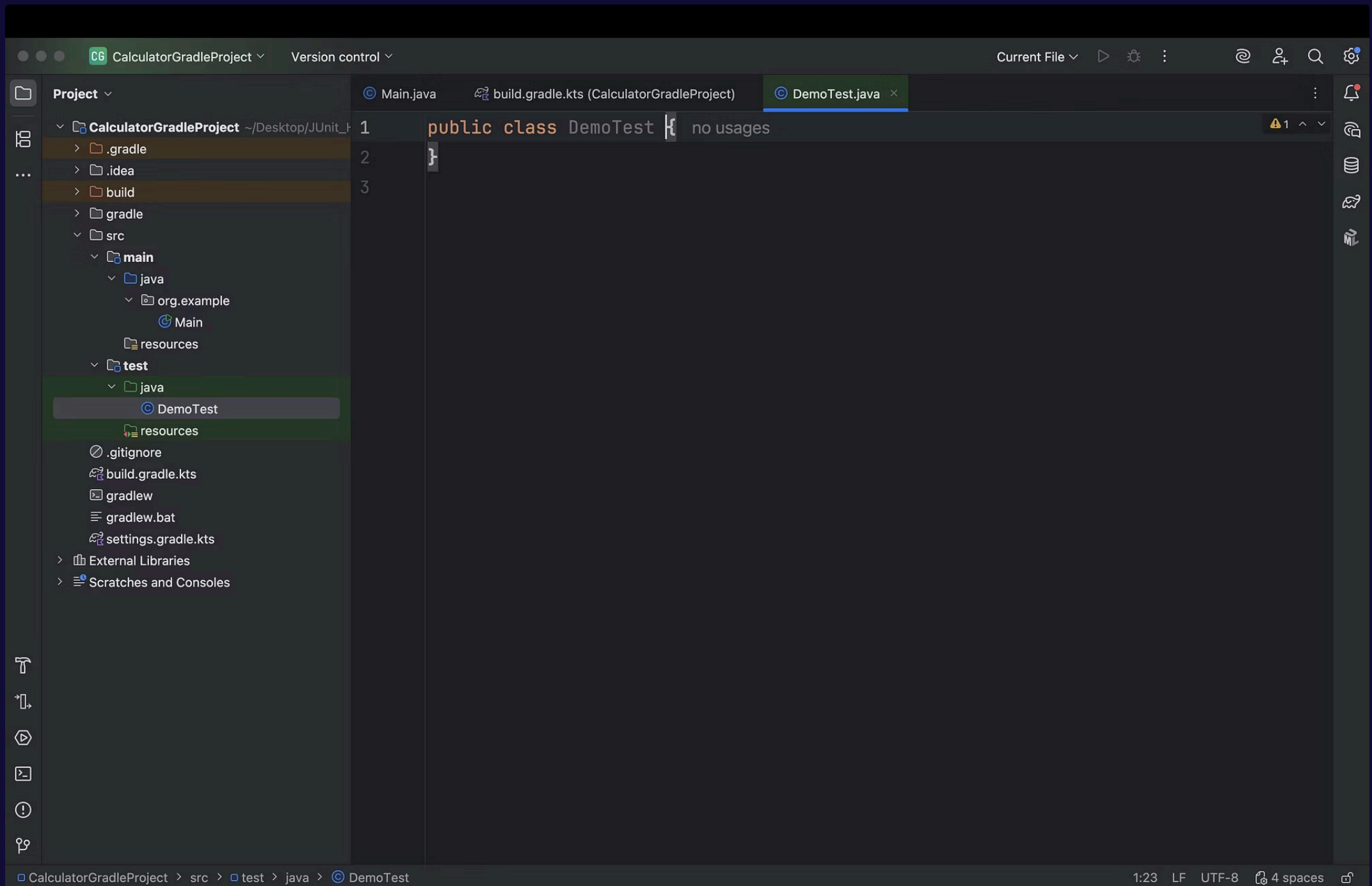
05

A green checkmark in the results panel confirms the test passed successfully

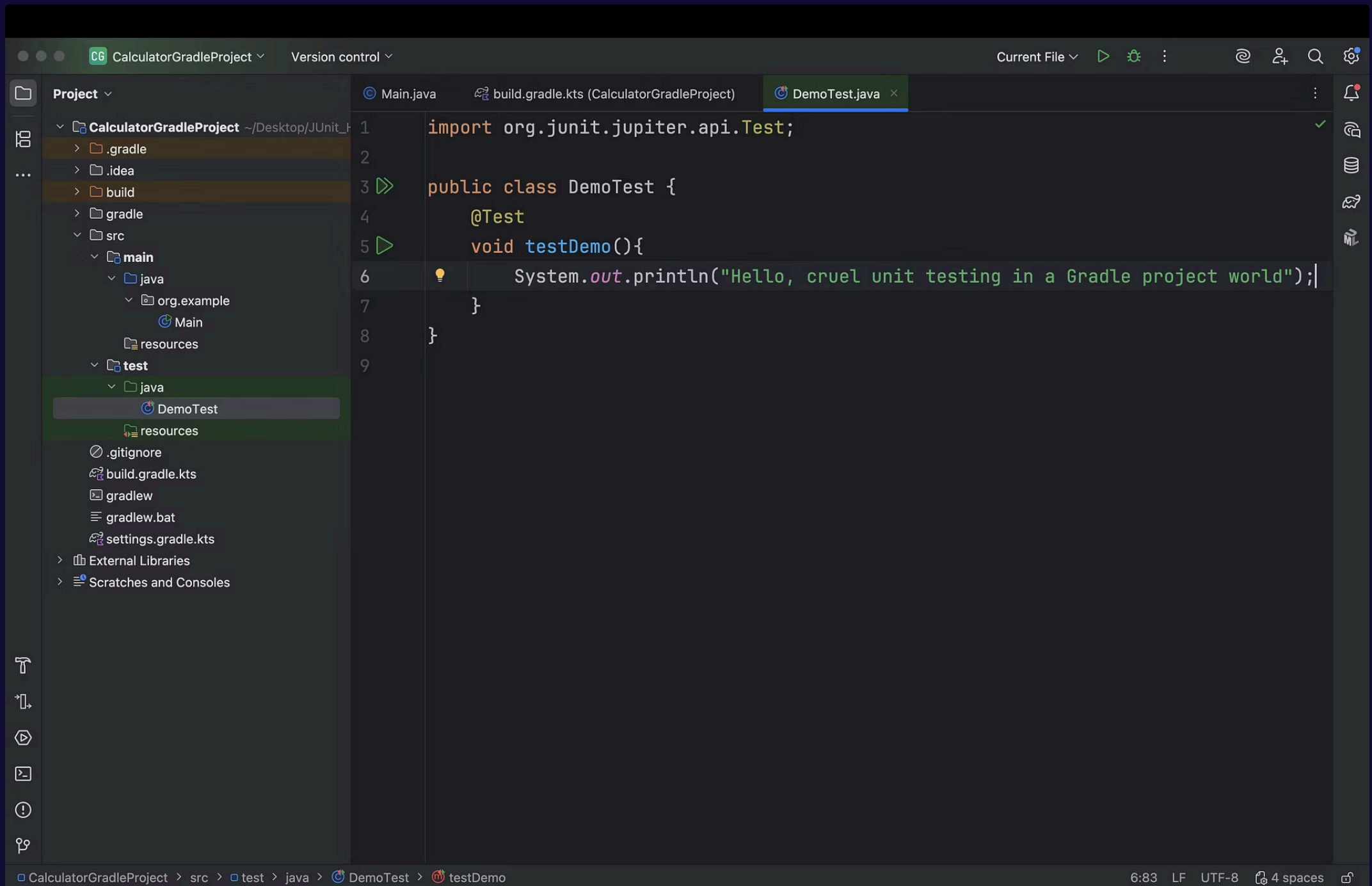
Creating a Simple Unit Test



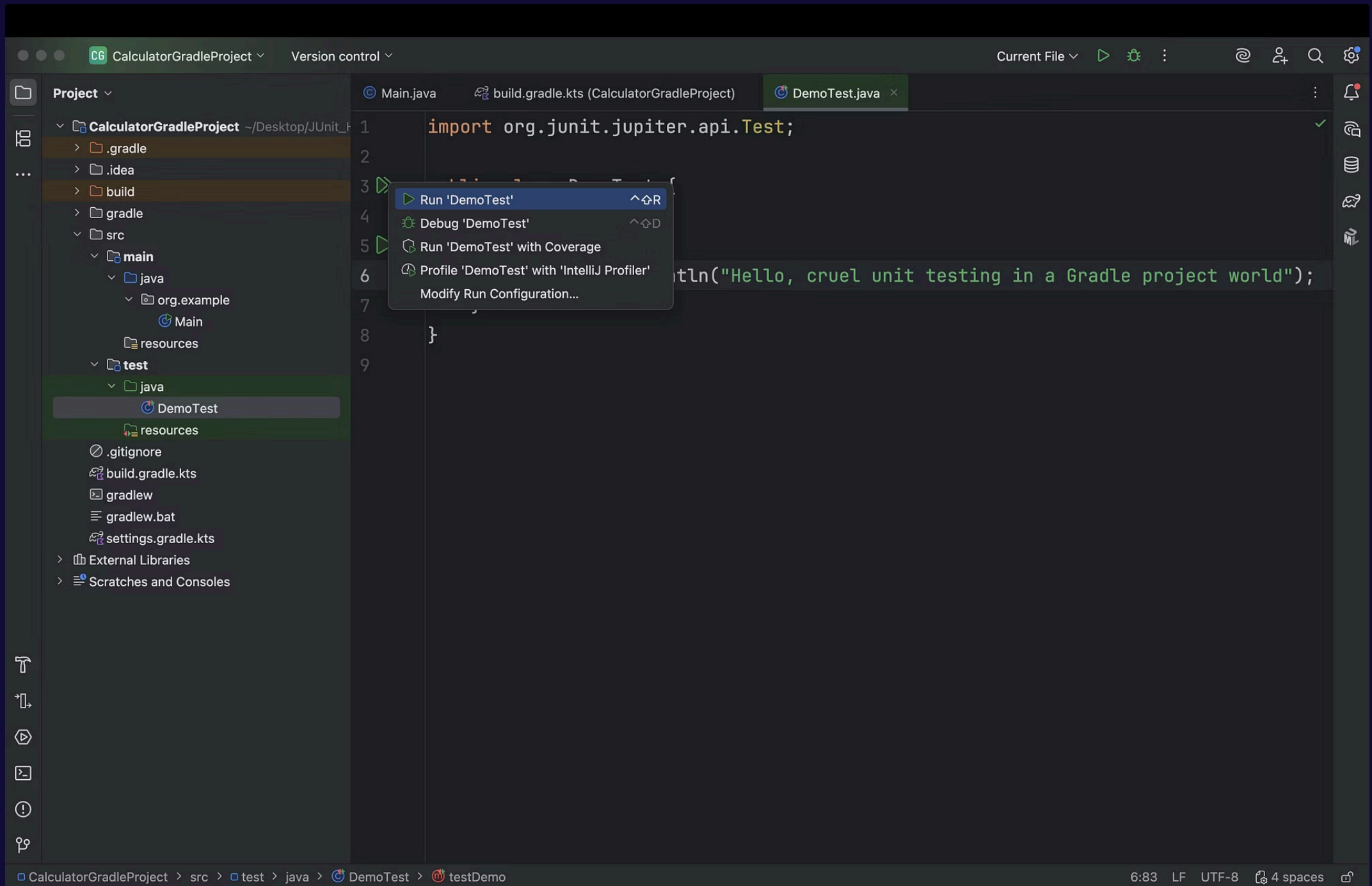
Creating a Simple Unit Test (contd.)



Creating a Simple Unit Test (contd.)



Running Tests Using IntelliJ



Running Tests Using IntelliJ (contd.)

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows the 'Run' button (a green play icon) and the 'Test' button (a green play icon with a magnifying glass). The 'Project' view on the left shows the project structure, with the 'test' directory expanded to show the 'DemoTest' class. The 'Main.java' file is open in the editor, showing the following code:

```
1 import org.junit.jupiter.api.Test;
2
3 public class DemoTest {
4     @Test
5     void testDemo(){
6         System.out.println("Hello, cruel unit testing in a Gradle project world");
7     }
8 }
9
```

The 'Run' view at the bottom shows the test results for 'DemoTest'. The test passed successfully, and the output console displays the following messages:

```
> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE
> Task :compileTestJava UP-TO-DATE
> Task :processTestResources NO-SOURCE
> Task :testClasses UP-TO-DATE
Hello, cruel unit testing in a Gradle project world
> Task :test
BUILD SUCCESSFUL in 1s
3 actionable tasks: 1 executed, 2 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.0
11:34:05: Execution finished ':test --tests "DemoTest"'.

```

The status bar at the bottom indicates the current file is 'CalculatorGradleProject > src > test > java > DemoTest > testDemo' and the encoding is UTF-8.

Running Tests Using Gradle Commands



Tests can also be executed using the Gradle wrapper in the terminal

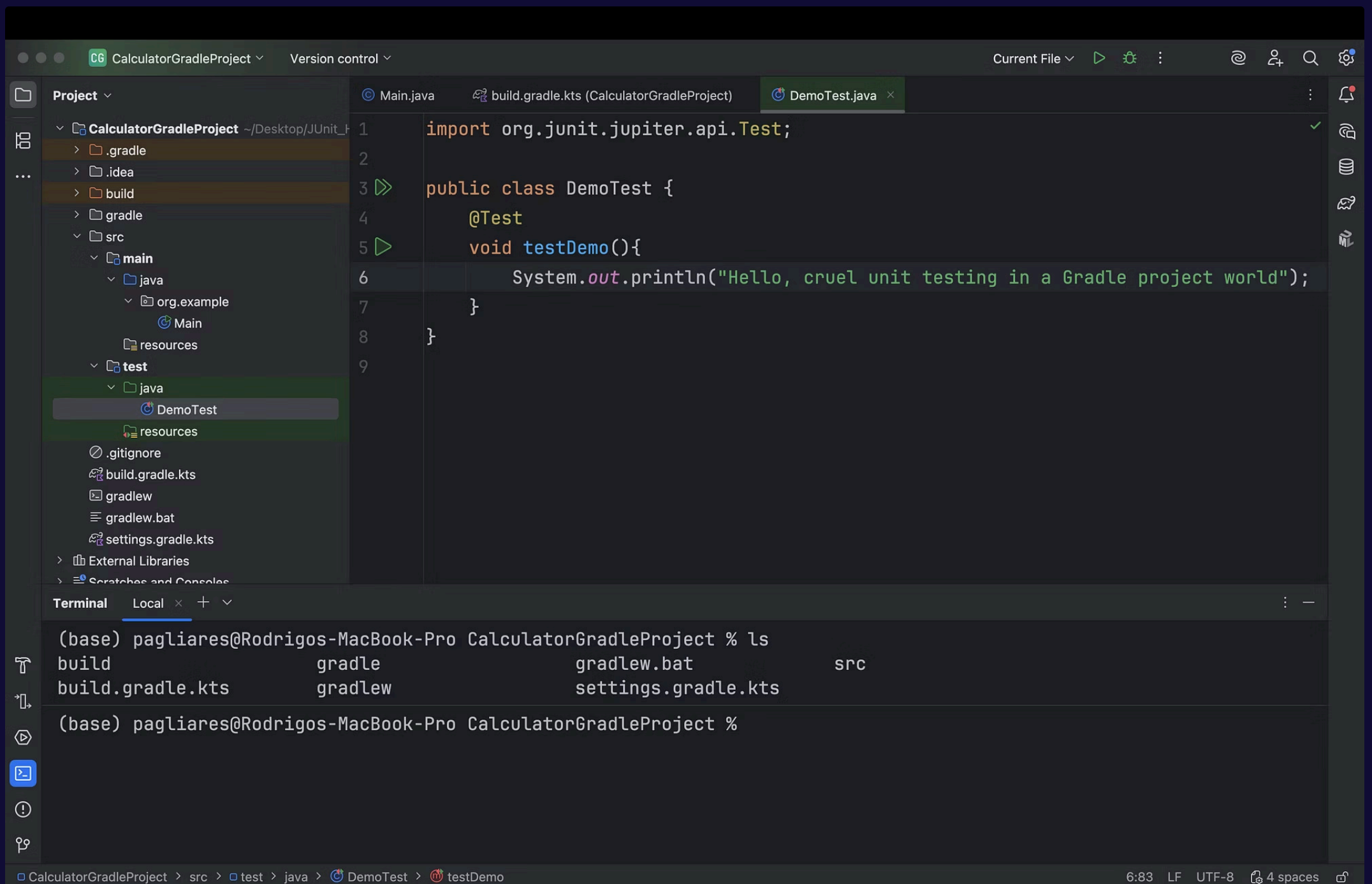
Ensure you're in the project's root directory (look for gradlew)

Run: `./gradlew clean test` to clean and execute the test task

Gradle will detect and run all test classes automatically

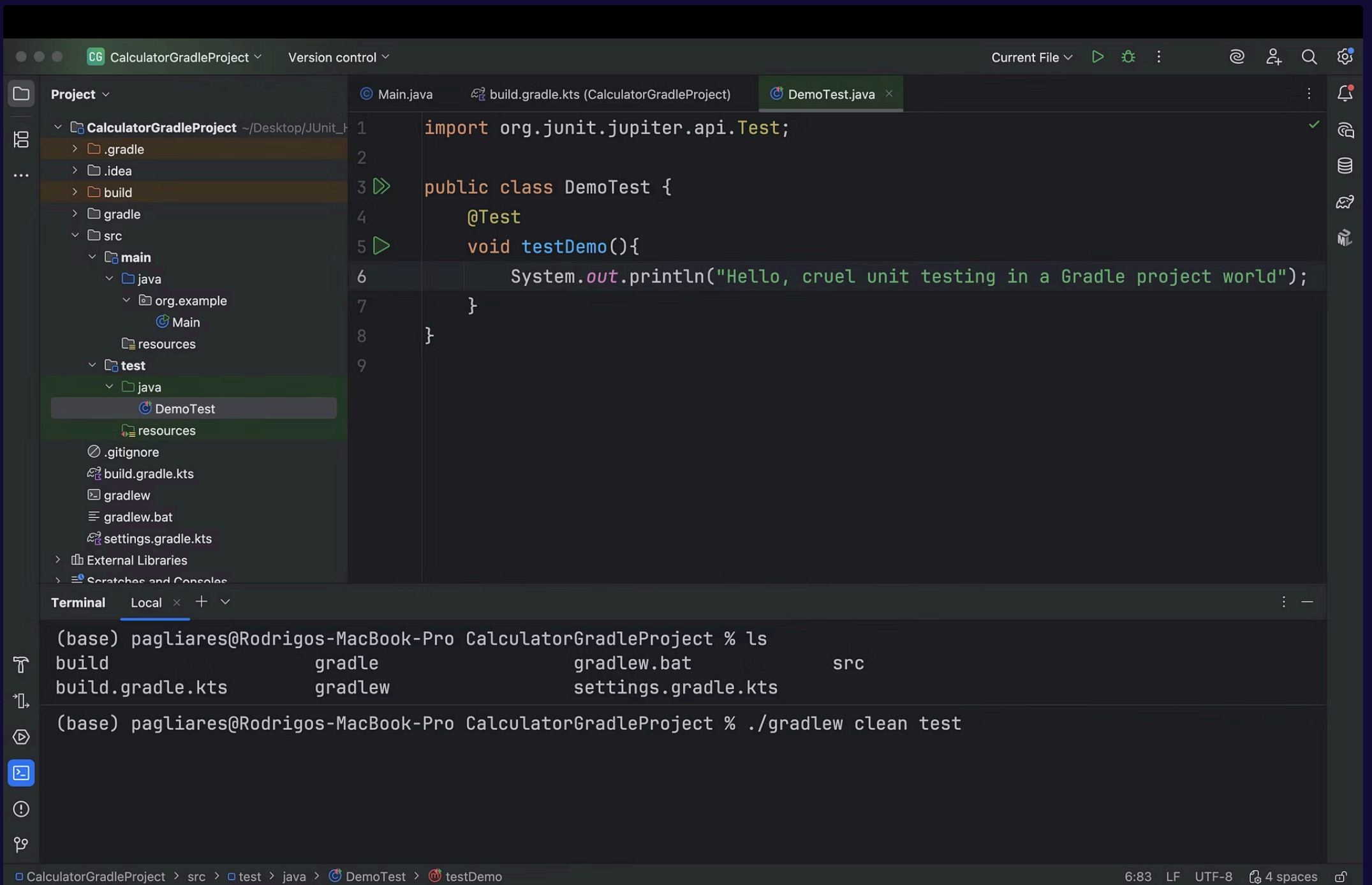
A "BUILD SUCCESSFUL" message confirms valid execution

Running the Simple Unit Test with gradlew



Open the **terminal window**.

Running the Simple Unit Test with gradlew (contd.)



The screenshot shows an IDE window for a project named "CalculatorGradleProject". The left sidebar displays the project structure, with the "test" directory expanded to show a "java" subdirectory containing the "DemoTest" class. The main editor area shows the code for "DemoTest.java":

```
1 import org.junit.jupiter.api.Test;
2
3 public class DemoTest {
4     @Test
5     void testDemo(){
6         System.out.println("Hello, cruel unit testing in a Gradle project world");
7     }
8 }
9
```

The bottom panel shows a terminal window with the following commands and output:

```
(base) pagliares@Rodrigos-MacBook-Pro CalculatorGradleProject % ls
build          gradle          gradlew.bat      src
build.gradle.kts  gradlew        settings.gradle.kts

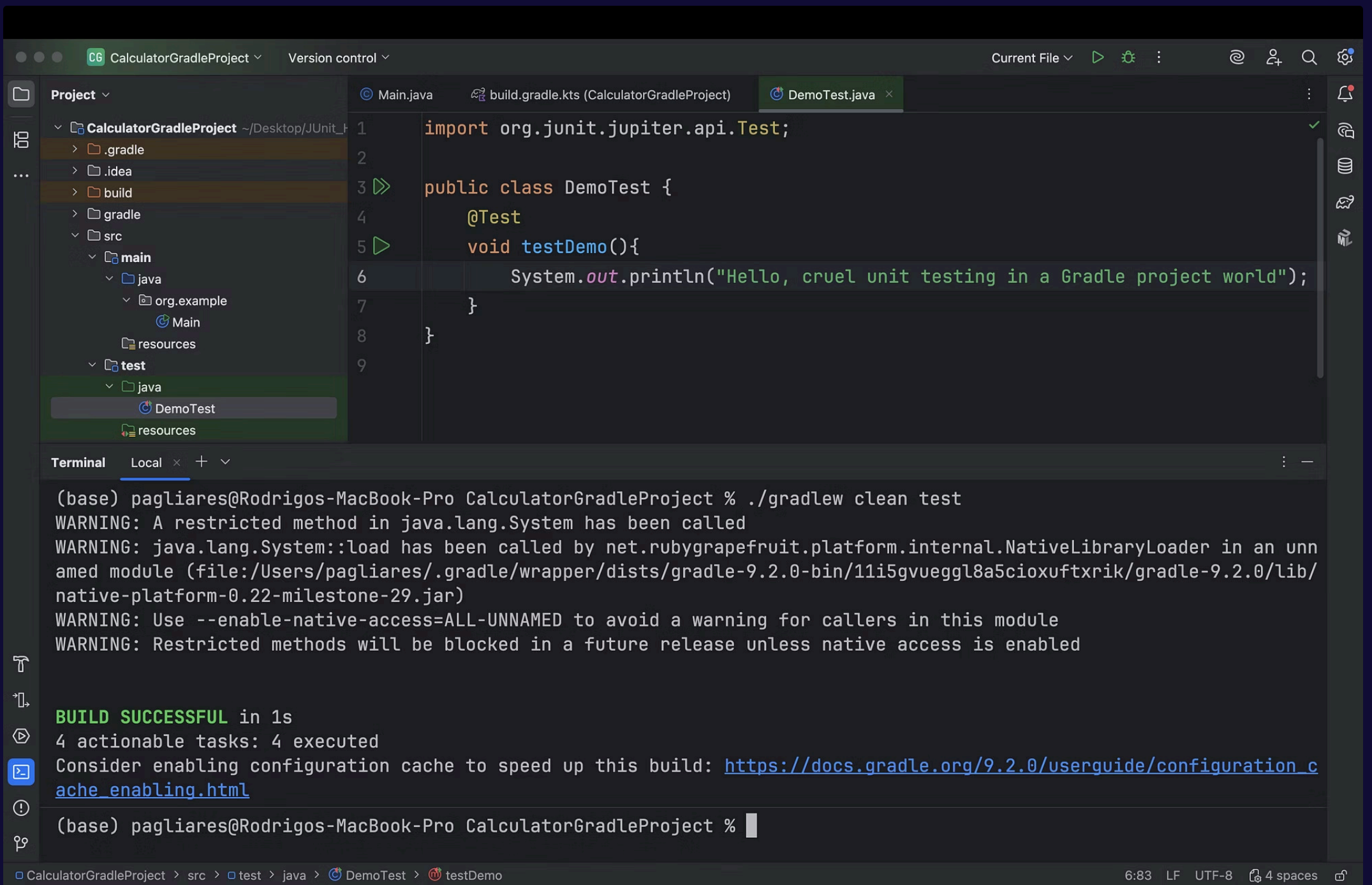
(base) pagliares@Rodrigos-MacBook-Pro CalculatorGradleProject % ./gradlew clean test
```

The status bar at the bottom indicates the current file path: "CalculatorGradleProject > src > test > java > DemoTest > testDemo", along with line 6, column 83, and encoding UTF-8.

Verify you are in the **root folder** of the **project**

Running the Simple Unit Test with gradlew (contd.)

Build successful! Gradle is now smoothly integrated with IntelliJ



The screenshot displays the IntelliJ IDEA interface for a project named 'CalculatorGradleProject'. The left sidebar shows the project structure with the 'test' directory selected. The main editor window shows the 'DemoTest.java' file with the following code:

```
1 import org.junit.jupiter.api.Test;
2
3 public class DemoTest {
4     @Test
5     void testDemo(){
6         System.out.println("Hello, cruel unit testing in a Gradle project world");
7     }
8 }
9
```

The bottom panel shows the 'Terminal' output for the command `./gradlew clean test`. The output indicates a successful build and test execution:

```
(base) pagliares@Rodrigos-MacBook-Pro CalculatorGradleProject % ./gradlew clean test
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by net.rubygrapefruit.platform.internal.NativeLibraryLoader in an unnamed module (file:/Users/pagliares/.gradle/wrapper/dists/gradle-9.2.0-bin/11i5gvuegg18a5cioxuftxrik/gradle-9.2.0/lib/native-platform-0.22-milestone-29.jar)
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

BUILD SUCCESSFUL in 1s
4 actionable tasks: 4 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.2.0/userguide/configuration\_cache\_enabling.html
(base) pagliares@Rodrigos-MacBook-Pro CalculatorGradleProject %
```

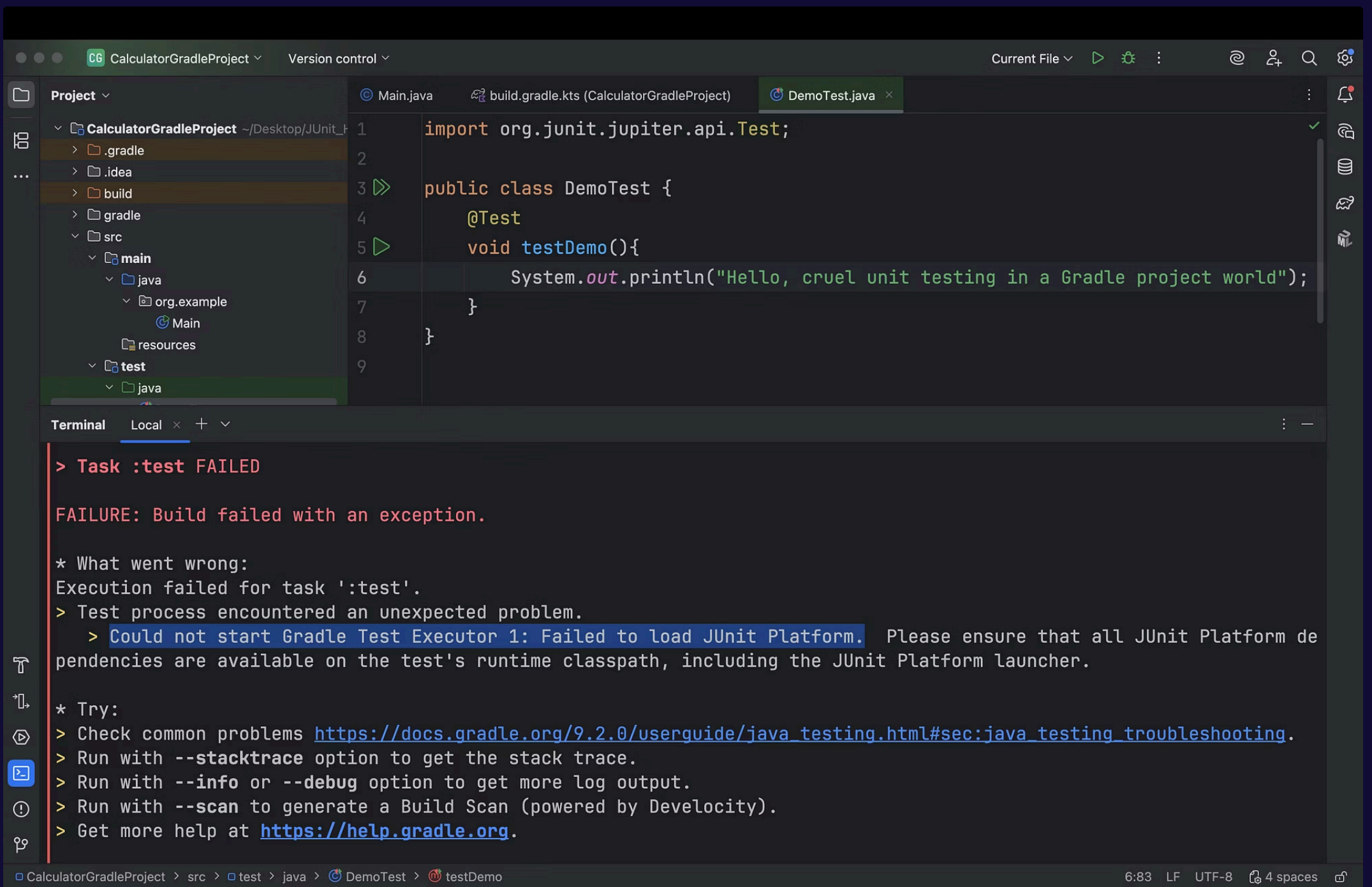
The status bar at the bottom indicates the current file is 'testDemo' and the encoding is UTF-8.

Build the project with the command `./gradlew clean test`

Troubleshooting!

NOTE: If you got the error message as in this printscreen, assure to include the dependency below in `build.gradle.kts`

```
// Required with Gradle 9+  
testRuntimeOnly("org.junit.platform:junit-platform-launcher")
```



The screenshot shows an IDE window for a project named "CalculatorGradleProject". The left sidebar displays the project structure, including folders for ".gradle", ".idea", "build", "gradle", "src", "main", "test", and "resources". The main editor area shows the file "DemoTest.java" with the following code:

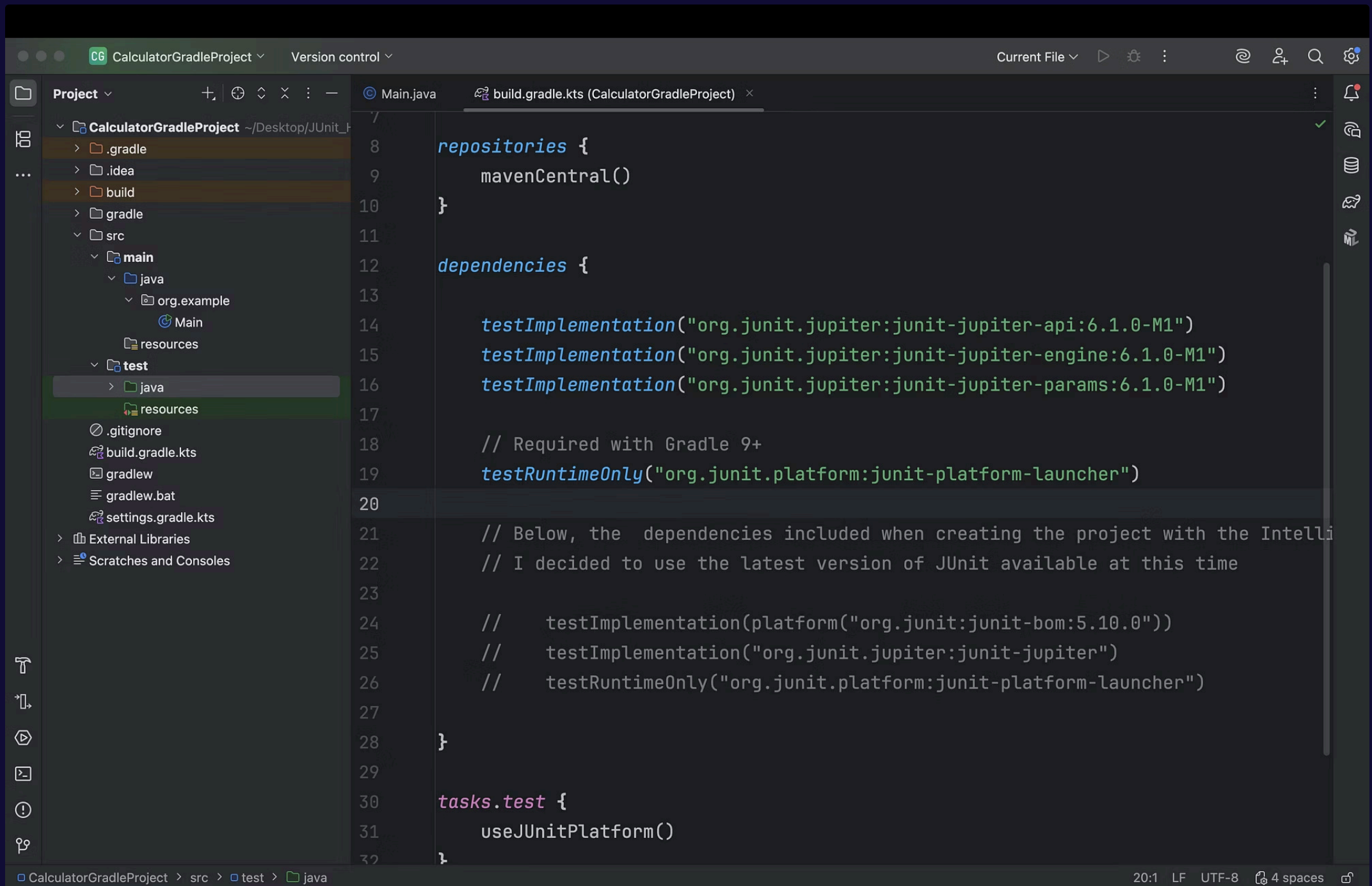
```
1 import org.junit.jupiter.api.Test;  
2  
3 public class DemoTest {  
4     @Test  
5     void testDemo(){  
6         System.out.println("Hello, cruel unit testing in a Gradle project world");  
7     }  
8 }  
9
```

The bottom panel shows the "Terminal" output, indicating a test failure:

```
> Task :test FAILED  
  
FAILURE: Build failed with an exception.  
  
* What went wrong:  
Execution failed for task ':test'.  
> Test process encountered an unexpected problem.  
    > Could not start Gradle Test Executor 1: Failed to load JUnit Platform. Please ensure that all JUnit Platform dependencies are available on the test's runtime classpath, including the JUnit Platform launcher.  
  
* Try:  
> Check common problems https://docs.gradle.org/9.2.0/userguide/java\_testing.html#sec:java\_testing\_troubleshooting.  
> Run with --stacktrace option to get the stack trace.  
> Run with --info or --debug option to get more log output.  
> Run with --scan to generate a Build Scan (powered by Develocity).  
> Get more help at https://help.gradle.org.
```

The status bar at the bottom indicates the current file is "DemoTest.java" and the test method is "testDemo".

Troubleshooting!



The screenshot shows an IDE window for a project named 'CalculatorGradleProject'. The left sidebar displays the project structure, with the 'test' directory selected. The main editor shows the 'build.gradle.kts' file. The code defines repositories, dependencies, and a test task. A comment indicates that the 'testRuntimeOnly' dependency is required for Gradle 9+.

```
7  
8 repositories {  
9     mavenCentral()  
10 }  
11  
12 dependencies {  
13  
14     testImplementation("org.junit.jupiter:junit-jupiter-api:6.1.0-M1")  
15     testImplementation("org.junit.jupiter:junit-jupiter-engine:6.1.0-M1")  
16     testImplementation("org.junit.jupiter:junit-jupiter-params:6.1.0-M1")  
17  
18     // Required with Gradle 9+  
19     testRuntimeOnly("org.junit.platform:junit-platform-launcher")  
20  
21     // Below, the dependencies included when creating the project with the IntelliJ  
22     // I decided to use the latest version of JUnit available at this time  
23  
24     // testImplementation(platform("org.junit:junit-bom:5.10.0"))  
25     // testImplementation("org.junit.jupiter:junit-jupiter")  
26     // testRuntimeOnly("org.junit.platform:junit-platform-launcher")  
27  
28 }  
29  
30 tasks.test {  
31     useJUnitPlatform()  
32 }
```

```
// Required with Gradle 9+  
testRuntimeOnly("org.junit.platform:junit-platform-launcher")
```

Confirming JUnit Support in Gradle Projects

Passing tests in both IDE and Gradle confirm proper configuration

Gradle prints test output when tests run from the terminal

This validates that dependencies and plugins are correctly applied

You are now ready to continue with more advanced test lessons

Both IDE-based and command-line testing workflows are fully functional



Conclusion

- JUnit 5 is correctly installed in the Gradle project
 - Tests run successfully in both the IDE and the Gradle command line
 - This dual validation ensures reliable test execution in all environments
 - You can now safely move forward to writing real-world unit tests
-

Open Floor Discussion

- Do you prefer running tests through your IDE or through Gradle?
- Have you faced issues where Gradle and the IDE behaved differently?
- How do you usually validate that your test environment is correctly configured?